Universidade Federal do Maranhão

Curso de Engenharia da Computação

PLANEJAMENTO DO PROJETO

Sistema Distribuído para Auxílio de Dados Hospitalares

Relatório de Implementação e Atualização do Cronograma: Marco 3

Curso: Engenharia da Computação

Disciplina: Sistemas Distribuídos

Professor(a): Luiz Henrique Neves Rodrigues

Grupo: Fernando da Silva Costa; Samara Santos Viegas; Eduardo dos Santos Oliveira; Herick Vinicius Pinheiro Da Conceição; Vyktor Gabryel Costa de Souza

14 de outubro de 2026

Sumário

1 Diagrama de Gantt e Atualização do Cronograma **1**

2 Introdução e Visão Geral **1**

3 Análise e Modelagem do Dataset Hospitalar **2**
3.1 Origem e Caracterização dos Dados 2
3.2 Mapeamento Semântico para a API 2

4 Recapitulação dos Marcos 1 e 2 **3**
4.1 Marco 1: Servidor Base e Ingestão de Dados 3
4.2 Marco 2: Cluster com Balanceador de Carga Round Robin 3
4.3 O Problema Identificado no Marco 2 (Motivação do Marco 3) 4

5 Marco 3 — Middleware MOM, Replicação e Escalabilidade **4**
5.1 Nova Topologia Integrada do Sistema Distribuído 4
5.2 Middleware Orientado a Mensagens (RabbitMQ) 5
5.3 Replicação Ativa Broadcast (Padrão Fanout) 6
5.4 Escalabilidade Horizontal Elástica (Scale-Out e Scale-In) 6
5.5 Visualização em Tempo Real e Acesso Remoto Seguro 7

6 Estado Atual da Replicação de Dados e Próximas Alterações **7**
6.1 O que já existe na arquitetura 7
6.2 O que ainda precisa ser desenvolvido 8
6.3 Critérios para considerar a replicação concluída 8

1 Diagrama de Gantt e Atualização do Cronograma

A Figura 1 ilustra o cronograma do projeto atualizado na data de **14 de outubro de 2026**. As barras em **verde** representam as etapas previamente finalizadas e homologadas (Planejamento, Ambiente e os Marcos 1 e 2). A barra em **amarelo** evidencia o **Marco 3 (Middleware, Replicação e Escalabilidade)**, objeto da presente entrega e em fase de consolidação. As barras em **cinza** delineiam as etapas futuras previstas para o Marco 4, baterias de injeção de falhas e encerramento.

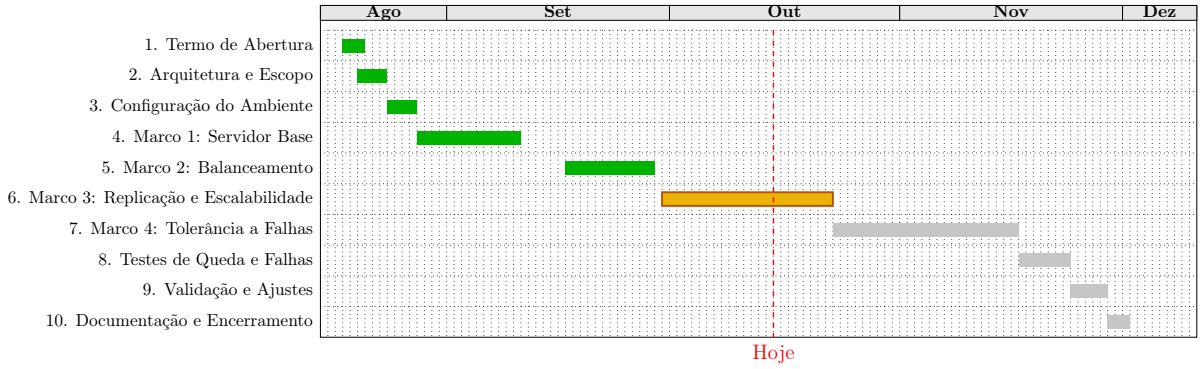


Figura 1: Diagrama de Gantt atualizado em 14/10/2026. Destaque em **amarelo** para a entrega do Marco 3.

Síntese da evolução cronológica:

- **Fases Concluídas (Verde):** Termo de abertura, levantamento de requisitos, arquitetura de rede, Marco 1 (API e SQLite) e Marco 2 (Load Balancer e cluster de 3 nós).
- **Fase Atual em Destaque (Amarelo): Marco 3**, contemplando a introdução do Middleware RabbitMQ, a estrutura de mensageria e a implementação pendente da replicação Fanout, além do planejamento do 4º servidor reserva em *Warm Standby* e o suporte a acesso público via túnel reverso.
- **Fases Subsequentes (Cinza):** O Marco 4 focará no endurecimento da tolerância a falhas ativas, chaveamento automático de falhas (failover sem intervenção humana) e encerramento documental.

2 Introdução e Visão Geral

Em ambientes hospitalares críticos, a telemetria contínua de sinais vitais de pacientes gera um fluxo massivo de dados em tempo real. Se esse tráfego converge para um único servidor centralizado, surgem gargalos severos de processamento, alta latência nas consultas clínicas e, fundamentalmente, a vulnerabilidade crítica de ponto único de falha (*Single Point of Failure* – SPoF). A indisponibilidade de tal sistema acarreta riscos à assistência médica. Com o objetivo de projetar, construir e validar uma solução robusta orientada aos conceitos de Sistemas Distribuídos, desenvolvemos o **Sistema Distribuído para Auxílio**

de Dados Hospitalares. O projeto adota uma metodologia evolutiva estruturada em marcos de maturidade crescente:

- **Marco 1 — Simulação e Servidor Base (Concluído):** Ingestão de dados reais de monitoramento biomédico (dataset clínico do Kaggle com 60.000 registros), validação formal de tipos através de modelos Pydantic na API RESTful (FastAPI) e persistência relacional local (SQLite).
- **Marco 2 — Balanceamento de Carga e Múltiplas Instâncias (Concluído):** Implantação de um cluster de três instâncias de aplicação independentes (:8001, :8002 e :8003) coordenadas por um Balanceador de Carga Round Robin (:8000), garantindo transparência de acesso e divisão estocasticamente uniforme do tráfego de entrada.
- **Marco 3 — Middleware Orientado a Mensagens (MOM), Replicação Ativa e Escalabilidade Elástica (Escopo Deste Relatório):** Evolução arquitetural com a introdução do RabbitMQ (AMQP 5672) para desacoplamento espaço-temporal, replicação de dados em broadcast *Fanout* para sincronização de 100% dos registros em todos os nós (eliminando a inconsistência e erros 404 do Marco 2), mecanismo de escalabilidade horizontal elástica (*Scale-Out* e *Scale-In*) com 4º nó em *Warm Standby* (:8004) e interface gráfica de visualização com suporte a túnel público seguro e fallback inteligente.

O projeto é integralmente implementado em Python (versão 3.10+) com tipagem estática e arquitetura modular desacoplada.

3 Análise e Modelagem do Dataset Hospitalar

3.1 Origem e Caracterização dos Dados

Os dados de telemetria derivam do dataset público do Kaggle:

`Synthetic_patient-HealthCare-Monitoring_dataset.csv`

A opção pelo uso de um repositório volumoso pré-existente viabiliza simulações fidedignas a cenários médicos reais. As principais características métricas são:

- **Volume:** 60.000 amostras clínicas ordenadas (60.001 linhas com o cabeçalho).
- **Integridade:** 0% de dados nulos ou registros ausentes em todas as variáveis.
- **Temporalidade:** O dataset não possui carimbo de tempo intrínseco. Assim, a camada de ingestão atribui o *timestamp* de alta precisão (ISO 8601) no momento da recepção em rede.

3.2 Mapeamento Semântico para a API

A Tabela 1 detalha o mapeamento semântico implementado no modelo Pydantic (`server/models.py`), convertendo colunas em inglês com caracteres especiais para tipos fortemente tipados.

Tabela 1: Mapeamento dos Atributos do CSV para o Esquema da API e Banco de Dados

Coluna Original (CSV)	Campo na API/DB	Tipo	Descrição Semântica
`Patient Number`	`paciente_id`	`int`	Identificador unívoco do paciente
`Heart Rate (bpm)`	`frequencia_cardiaca`	`int`	Frequência cardíaca (bpm)
`SpO2 Level (%)`	`spo2`	`int`	Saturação de oxigênio periférico
`Systolic Blood Pressure`	`pressao_sistolica`	`int`	Pressão arterial sistólica (mmHg)
`Diastolic Blood Pressure`	`pressao_diastolica`	`int`	Pressão arterial diastólica (mmHg)
`Body Temperature (°C)`	`temperatura`	`float`	Temperatura corpórea (°C)
`Fall Detection`	`deteccao_queda`	`str`	Sensor de queda (Yes/No)
`Predicted Disease`	`doenca_prevista`	`str`	Diagnóstico clínico correlacionado
`Data Accuracy (%)`	`precisao_dados`	`int`	Índice de confiabilidade do sensor
`Heart Rate Alert`	`alerta_freq_cardiaca`	`str`	Alerta de FC (NORMAL/ABNORMAL)
`SpO2 Level Alert`	`alerta_spo2`	`str`	Alerta clínico de hipóxia
`Blood Pressure Alert`	`alerta_pressao`	`str`	Alerta de hipertensão/hipotensão
`Temperature Alert`	`alerta_temperatura`	`str`	Alerta térmico (hipo/hipertermia)

4 Recapitulação dos Marcos 1 e 2

4.1 Marco 1: Servidor Base e Ingestão de Dados

No Marco 1, consolidou-se o fluxo unitário de transmissão e armazenamento de telemetria médica. O componente leitor (`client/csv_sender.py`) lê o dataset, converte as linhas para payloads JSON estruturados e os submete via requisições HTTP `POST /dados` a um servidor base FastAPI rodando na porta 8001. O servidor valida os dados através do schema Pydantic, injeta metadados (timestamp e identificador `server-01`) e persiste o registro no banco relacional local SQLite (`server1.db`), conforme esquematizado na Figura 2.



Figura 2: Arquitetura do Marco 1: Ingestão direta e persistência mononó.

4.2 Marco 2: Cluster com Balanceador de Carga Round Robin

No Marco 2, o sistema escalou para uma topologia com três instâncias idênticas e isoladas da aplicação executando nas portas 8001, 8002 e 8003. Para distribuir o tráfego de maneira transparente, foi introduzido o Balanceador de Carga (`load_balancer/balancer.py`) na porta 8000, implementando o algoritmo Round Robin clássico sobre o conjunto $\mathcal{S} = s_0, s_1, s_2$:

$$\text{Servidor Alvo}(r) = s_r \pmod 3 \tag{1}$$

A Figura 3 ilustra a topologia do Marco 2. Cada servidor gravava exclusivamente no seu respectivo banco SQLite local (`server1.db`, `server2.db` ou `server3.db`).

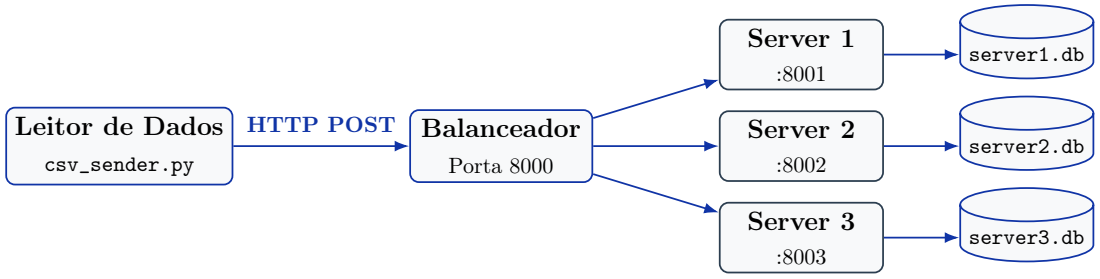


Figura 3: Topologia do Marco 2: Balanceamento Round Robin entre 3 nós independentes.

4.3 O Problema Identificado no Marco 2 (Motivação do Marco 3)

Embora o Marco 2 tenha resolvido a divisão homogênea da carga de escrita, ele revelou uma limitação crítica de consistência em sistemas distribuídos: **o isolamento estrito dos bancos de dados locais**. Como cada servidor operava como uma ilha de dados, ao enviar 12 requisições, cada banco retinha apenas 4 registros intercalados. Consequentemente:

1. **Inconsistência de Leitura e Erros 404:** Uma requisição clínica `GET /dados/paciente_id`
   direcionada pelo balanceador para o Servidor 02 retornava `HTTP 404 Not Found` caso a medição daquele paciente tivesse sido processada pelo Servidor 01 ou 03.

2. **Falta de Desacoplamento Temporal:** O emissor dependia da disponibilidade síncrona dos servidores HTTP. Se o cluster estivesse temporariamente saturado, medições de pacientes eram descartadas.

3. **Rigidez Operacional:** O cluster não previa adição dinâmica de capacidade computacional sem reinicialização do balanceador.

Essas deficiências fundamentaram o escopo de inovações desenvolvidas no **Marco 3**.

5 Marco 3 — Middleware MOM, Replicação e Escalabilidade

O Marco 3 introduz componentes de mensageria e prepara a arquitetura para replicação broadcast, tolerância a picos de tráfego e escalabilidade elástica. A replicação efetiva entre os três servidores permanece pendente.

5.1 Nova Topologia Integrada do Sistema Distribuído

A Figura 4 apresenta a topologia arquitetural planejada para o Marco 3. A presença de consumidores e conexões no diagrama representa o desenho pretendido, não uma comprovação de que a replicação já esteja concluída.



Figura 4: Topologia visual do sistema distribuído, conforme a interface de monitoramento. A representação ilustra os componentes e o fluxo arquitetural planejado; a replicação efetiva dos dados entre os servidores permanece pendente de implementação e validação.

5.2 Middleware Orientado a Mensagens (RabbitMQ)

No contexto de Sistemas Distribuídos, o middleware MOM (*Message-Oriented Middleware*) atua como o mediador desacoplador entre produtores e consumidores. Seus benefícios fundamentais comprovados no projeto são:

- **Desacoplamento Espacial:** O produtor desconhece os endereços IP, portas ou o quantitativo de servidores em execução.
- **Desacoplamento Temporal:** As mensagens são persistidas no broker. Caso os servidores estejam temporariamente desligados ou processando lotes pesados, as telemetrias aguardam nas filas sem perda de pacotes.
- **Confiabilidade e Garantia de Entrega:** Configuração de mensagens persistentes (`delivery_mode=2`) combinada com confirmações explícitas de processamento (`basic_ack` e `basic_nack` com `requeue=True`).

O módulo `middleware/config.py` centraliza a topologia AMQP:

```
EXCHANGE_TELEMETRIA = "telemetria_hospitalar"
TIPO_EXCHANGE = "fanout"
FILAS_SERVIDORES = [
    "fila_servidor_1",
```

```
    "fila_servidor_2",
    "fila_servidor_3",
    "fila_servidor_4",
]
```

Listing 1: Definição da Infraestrutura AMQP em middleware/config.py

5.3 Replicação Ativa Broadcast (Padrão Fanout)

Para solucionar a inconsistência de dados do Marco 2, adotamos o padrão de **Replicação Ativa via Exchange Fanout**. Quando o produtor publica uma medição na exchange `telemetria_hospitalar`, o RabbitMQ clona e entrega instantaneamente a mensagem para **todas as quatro filas** simultaneamente. O consumidor deverá ser integrado e validado para instanciar consumidores independentes destinados às filas de cada servidor. Cada consumidor drena sua respectiva fila e persiste a medição no banco SQLite correspondente:

- Consumidor 1 → `fila_servidor_1` → `server1.db`
- Consumidor 2 → `fila_servidor_2` → `server2.db`
- Consumidor 3 → `fila_servidor_3` → `server3.db`
- Consumidor 4 → `fila_servidor_4` → `server4.db`

Impacto esperado na consistência: após a implementação e validação, os bancos deverão conter os mesmos registros. Até lá, não se pode garantir que qualquer consulta `GET /dados/`
`paciente_id`
direcionada pelo balanceador para qualquer servidor resulta em `HTTP 200 OK` imediato com o histórico clínico completo.

5.4 Escalabilidade Horizontal Elástica (Scale-Out e Scale-In)

Uma das inovações do Marco 3 é o suporte à elasticidade dinâmica de recursos computacionais:

- **O Conceito de Warm Standby:** O Servidor 04 (:8004) opera continuamente sincronizado em segundo plano, recebendo as réplicas do middleware no banco `server4.db`. Porém, no estado inicial, ele não recebe tráfego HTTP direto de clientes.
- **Scale-Out Dinâmico (Ativar Reserva):** Quando o cluster sofre picos de carga, o operador aciona a rota `POST /servidores/ativar-reserva` no balanceador. Instantaneamente, o balanceador promove o Servidor 04 para o anel ativo. Como o banco `server4.db` já estava replicado em tempo real, o nó entra em operação imediatamente, sem latência de sincronização inicial. O algoritmo Round Robin recalcula automaticamente o anel para $N = 4$ servidores:

$$\text{Servidor Alvo}(r) = s_r \pmod 4 \tag{2}$$

  A carga unitária por servidor é reduzida de 33,3% para 25,0%.

- **Scale-In Dinâmico (Desativar Reserva):** Cessada a sobrecarga, a rota `POST /servidores/desativar-reserva` remove o Servidor 04 do anel ativo de roteamento HTTP, retornando com transparência e zero downtime para $N = 3$ servidores.

5.5 Visualização em Tempo Real e Acesso Remoto Seguro

Para demonstrações acadêmicas e validação operacional, o Marco 3 integrou:

- **Monitor Web Interativo (:5000):** Interface moderna (`visualizacao/monitor.py`) provida de animações SVG dos pacotes trafegando entre nós, métricas de distribuição em tempo real e console de eventos.
- **Túnel Público Seguro via Cloudflare (`tunel.sh`):** Integração de túnel reverso criptografado HTTPS (`trycloudflare.com`), permitindo que bancas avaliadoras e dispositivos móveis externos (4G/5G) acessem a interface diretamente do PC do desenvolvedor sem necessidade de port forwarding ou abertura de portas em roteadores.
- **Mecanismo de Smart Polling Fallback:** Solução de engenharia desenvolvida para superar o buffering agressivo que CDNs e operadoras móveis aplicam em conexões HTTP streaming (Server-Sent Events). Se o stream SSE for retido, a interface chaveia em 350ms para o endpoint `/api/eventos-poll`, garantindo animação contínua e sem travamentos.

6 Estado Atual da Replicação de Dados e Próximas Alterações

Embora a arquitetura já contemple o produtor de mensagens, o RabbitMQ, a exchange do tipo *Fanout*, as filas dedicadas e os três servidores principais, **a replicação correta e efetiva dos dados entre os três servidores ainda não foi desenvolvida e validada integralmente**. Portanto, não é correto afirmar, nesta etapa, que os bancos `server1.db`, `server2.db` e `server3.db` mantêm cópias idênticas dos registros. O balanceador de carga distribui as requisições entre as instâncias, mas isso, por si só, não replica os dados. Como cada instância utiliza seu próprio banco SQLite local, um registro gravado por um servidor pode não estar disponível quando uma consulta posterior for encaminhada a outro servidor. Essa limitação pode ocasionar resultados incompletos ou respostas `HTTP 404`, dependendo da rota e do identificador consultado.

6.1 O que já existe na arquitetura

- Três instâncias FastAPI principais, nas portas 8001, 8002 e 8003.
- Balanceador de carga na porta 8000, com distribuição Round Robin.
- Estrutura de middleware RabbitMQ e definição arquitetural de exchange *Fanout* e filas destinadas aos consumidores.
- Bancos SQLite locais associados às instâncias.

- Representação visual da topologia e do servidor reserva na interface de monitoramento.

A existência dos componentes de infraestrutura não significa, por si só, que o fluxo completo de replicação esteja concluído. A publicação das mensagens, o consumo independente por cada servidor, a gravação persistente e a verificação de igualdade dos dados precisam funcionar de ponta a ponta.

6.2 O que ainda precisa ser desenvolvido

1. Implementar e integrar o consumidor de mensagens em cada um dos três servidores, garantindo que cada registro publicado seja recebido e persistido em todas as réplicas pretendidas.

2. Definir uma estratégia idempotente para impedir duplicações quando uma mensagem for entregue novamente.

3. Confirmar o processamento da mensagem (ACK) somente depois da gravação bem-sucedida no banco correspondente; em caso de falha, tratar a rejeição ou a reentrega sem perder dados.

4. Definir identificadores estáveis para os registros e regras de conflito, para que a repetição de mensagens não produza inconsistências.

5. Testar a consistência comparando a quantidade e o conteúdo dos registros nos três bancos após o envio de um conjunto conhecido de dados.

6. Testar indisponibilidade temporária de um consumidor e verificar se as mensagens pendentes são processadas após a recuperação.

6.3 Critérios para considerar a replicação concluída

A replicação só deverá ser considerada implementada após testes reproduzíveis demonstrarem que, para um conjunto de registros publicado, os três bancos contêm os mesmos registros, sem omissões e sem duplicações indevidas. Também deverá ser verificado que falhas temporárias de consumidores não causam perda silenciosa de mensagens. Até que esses critérios sejam atendidos, a replicação deve ser descrita como **pendente de implementação e validação**, e não como uma funcionalidade concluída. O servidor reserva e os mecanismos de escalabilidade também não devem ser apresentados como sincronizados em tempo real sem evidências experimentais que comprovem esse comportamento.